

## 14. Flyweight Design Pattern in Java

### 14.1 Introduction

The **Flyweight Design Pattern** is a **structural pattern** used to reduce memory consumption by sharing common object data instead of duplicating it.

### 14.2 Problem it Solves

When many objects share common data, creating duplicate instances leads to high memory usage. Flyweight minimizes memory by reusing existing objects.

### 14.3 What is Flyweight Pattern?

Flyweight pattern ensures that a class has only one instance per unique state (intrinsic state), allowing shared use across different contexts (with extrinsic state).

### 14.4 Real-World Analogy

A font rendering system stores only one object per character style (e.g., Arial 12pt) and reuses it for every letter drawn.

### 14.5 Structure

```
package com.mannemlokes;

import java.util.HashMap;
import java.util.Map;

interface Flyweight {
    void operation(String extrinsicState);
}

class ConcreteFlyweight implements Flyweight {
    private final String intrinsicState;

    public ConcreteFlyweight(String intrinsicState) {
        this.intrinsicState = intrinsicState;
    }

    public void operation(String extrinsicState) {
        System.out.println("Using Intrinsic: " + intrinsicState + ",
                           Extrinsic: " + extrinsicState);
    }
}

public class FlyweightFactory {
    private Map<String, Flyweight> pool = new HashMap<>();

    public Flyweight getFlyweight(String key) {
        if (!pool.containsKey(key)) {
            pool.put(key, new ConcreteFlyweight(key));
        }
        return pool.get(key);
    }
}
```

## 14.6 Benefits

- Drastically reduces memory footprint
- Improves performance when dealing with large numbers of similar objects

## 14.7 Implementation Steps

1. Separate intrinsic and extrinsic state
2. Create flyweight objects to store intrinsic state
3. Use a factory to manage flyweight objects
4. Pass extrinsic state at runtime

## 14.8 Examples

### Example1: Character Objects

```
package com.mannemlokes;

import java.util.HashMap;
import java.util.Map;

interface Character {
    void display(String color);
}

class ConcreteCharacter implements Character {
    private final char symbol;

    public ConcreteCharacter(char symbol) {
        this.symbol = symbol;
    }

    public void display(String color) {
        System.out.println("Displaying '" + symbol + "' in color: " + color);
    }
}

class CharacterFactory {
    private static final Map<java.lang.Character, com.mannemlokes.Character>
        characters = new HashMap<>();

    public static Character getCharacter(char c) {
        characters.putIfAbsent(c, new ConcreteCharacter(c));
        return characters.get(c);
    }
}

public class Example1CharacterClient {
    public static void main(String[] args) {
        Character c1 = CharacterFactory.getCharacter('A');
        Character c2 = CharacterFactory.getCharacter('B');
        Character c3 = CharacterFactory.getCharacter('A');

        c1.display("Red");
        c2.display("Green");
        c3.display("Blue");
    }
}
```

---



---

```
}
```

**Output:**

```
Displaying 'A' in color: Red
Displaying 'B' in color: Green
Displaying 'A' in color: Blue
```

**Example2: Tree Rendering in a Game**

```
package com.mannemlokes;

import java.util.HashMap;
import java.util.Map;

class TreeType {
    private String name, color, texture;

    public TreeType(String name, String color, String texture) {
        this.name = name;
        this.color = color;
        this.texture = texture;
    }

    public void draw(int x, int y) {
        System.out.println("Drawing " + name + "tree with " + color + "
                           color and " + texture + " texture " + "at (" + x
                           + ", " + y + ")");
    }
}

class TreeFactory {
    private static Map<String, TreeType> treeTypes = new HashMap<>();

    public static TreeType getTreeType(String name, String color, String
                                       texture) {
        String key = name + color + texture;
        treeTypes.putIfAbsent(key, new TreeType(name, color, texture));
        return treeTypes.get(key);
    }
}

class Tree {
    private int x, y;
    private TreeType type;

    public Tree(int x, int y, TreeType type) {
        this.x = x;
        this.y = y;
        this.type = type;
    }

    public void draw() {
        type.draw(x, y);
    }
}

public class Example2Forest {
```

```

public static void main(String[] args) {
    TreeType oak = TreeFactory.getTreeType("Oak", "Green", "Rough");
    TreeType pine = TreeFactory.getTreeType("Pine", "Dark Green",
                                            "Smooth");

    Tree tree1 = new Tree(10, 20, oak);
    Tree tree2 = new Tree(30, 40, pine);
    Tree tree3 = new Tree(50, 60, oak);

    tree1.draw();
    tree2.draw();
    tree3.draw();
}
}

```

**Output:**

```

Drawing Oaktree with Green color and Rough texture at (10, 20)
Drawing Pinetree with Dark Green color and Smooth texture at (30, 40)
Drawing Oaktree with Green color and Rough texture at (50, 60)

```

**Example3: Database Connection Pooling (Simulation)**

```

package com.mannemlokes;

import java.util.HashMap;
import java.util.Map;

class DBConnection {
    private String dbURL;

    public DBConnection(String dbURL) {
        this.dbURL = dbURL;
        System.out.println("Connecting to: " + dbURL);
    }

    public void execute(String query) {
        System.out.println("Executing: " + query + " on " + dbURL);
    }
}

class ConnectionPool {
    private static Map<String, DBConnection> pool = new HashMap<>();

    public static DBConnection getConnection(String dbURL) {
        pool.putIfAbsent(dbURL, new DBConnection(dbURL));
        return pool.get(dbURL);
    }
}

public class Example3DBClient {
    public static void main(String[] args) {
        DBConnection conn1 =
            ConnectionPool.getConnection("jdbc:mysql://localhost:3306/db1");
        conn1.execute("SELECT * FROM users");

        DBConnection conn2 =
            ConnectionPool.getConnection("jdbc:mysql://localhost:3306/db2");
    }
}

```

```

        conn2.execute("SELECT * FROM orders");

        DBConnection conn3 =
            ConnectionPool.getConnection("jdbc:mysql://localhost:3306/db1");
        conn3.execute("SELECT * FROM payments");
    }
}

```

**Output:**

```

Connecting to: jdbc:mysql://localhost:3306/db1
Executing: SELECT * FROM users on jdbc:mysql://localhost:3306/db1
Connecting to: jdbc:mysql://localhost:3306/db2
Executing: SELECT * FROM orders on jdbc:mysql://localhost:3306/db2
Connecting to: jdbc:mysql://localhost:3306/db1
Executing: SELECT * FROM payments on jdbc:mysql://localhost:3306/db1

```

## 14.9 Intrinsic vs Extrinsic State

| Type      | Description                        | Example                     |
|-----------|------------------------------------|-----------------------------|
| Intrinsic | Shared, stored in Flyweight object | Character symbol, tree type |
| Extrinsic | Passed from outside at runtime     | Font color, tree location   |

## 14.10 Flyweight vs Other Patterns

| Pattern     | Purpose                                 |
|-------------|-----------------------------------------|
| Flyweight   | Reduce memory by sharing common state   |
| Singleton   | Ensure one instance globally            |
| Prototype   | Clone existing object                   |
| Object Pool | Reuse pre-initialized expensive objects |

## 14.11 Pros and Cons

### ✓ Pros

- Reduced memory footprint
- Improved performance for repetitive data

### ✗ Cons

- Increased complexity due to external state
- Need to manage object sharing carefully

## 14.12 When to Use / Not to Use

### Use When:

- You have many objects with shared data
- Memory is a concern (graphics, documents, caching)

### Avoid When:

- Objects are highly unique
- Overhead of managing extrinsic state is high

### 14.13 Summary

| Feature     | Description                            |
|-------------|----------------------------------------|
| Intent      | Minimize memory by sharing data        |
| Use Case    | GUIs, games, fonts, databases          |
| Key Benefit | Efficient memory and performance usage |

The **Flyweight Pattern** is essential in memory-critical applications, enabling large-scale object reuse with shared intrinsic state.